Table des matières

Cours 03 — Monter le projet dbt et le STAGING 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Le dépôt et le projet · STAGING 2
Étape 2 — Déclarer la RAW · STAGING 5
Étape 3 — La première vue, stg_tickets · STAGING 6
Étape 4 — Bâtir la vue, puis la lire · STAGING 9
Étape 5 — Les trois autres vues · STAGING 13
Étape 6 — Bâtir les trois vues, puis les lire · STAGING 15
Étape 7 — Versionner · STAGING 17
À retenir 17

Cours 03 — Monter le projet dbt et le STAGING

Cours 3/9 · La Brûlerie — Formation Analytics Engineering

Data Engineering	**Analytics Engineering**		
RAW	STAGING	INTERMEDIATE	MARTS
chargée telle que livrée →	reprendre la RAW, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
	✓ ici		

Ce que vous allez faire

dbt est l'outil de l'ingénieur analytique : vous écrivez des `select`, et il en fait des vues et des tables dans l'entrepôt, dans le bon ordre. Chaque `select` est un modèle, rangé dans un fichier `.sql`. Vous montez le projet dbt de La Brûlerie, `02_ae`, un dépôt Git à côté des sources, puis vous y écrivez les quatre vues du `STAGING`, une par table que l'étoile lira : `stg_tickets` d'abord, qui rend les tickets lisibles, puis `stg_lignes`, `stg_produits` et `stg_membres`. Une vue est une requête enregistrée sous un nom : elle ne garde aucune donnée, elle relit la `RAW` chaque fois qu'on l'interroge.

© 2026 iAlexMG. Tous droits réservés.

la RAW → source() → quatre vues du STAGING, dans l'entrepôt partagé

Le STAGING a quatre gestes, et une règle : rien de personnel ne monte. Les tickets demandent deux gestes, typer et tirer l'état courant ; les trois autres vues montrent le reste. Les fichiers à écrire se donnent en pseudocode, et le code complet est dans Codes_03_AE.md, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans Projet_Brulerie :

- la première pour le Bloc-notes, dbt et Git ;
- la seconde pour le notebook de DuckDB (duckdb -ui), où se tapent les requêtes ; elle s'ouvre à l'étape 4.

Deux endroits, parce que deux usages. Ce qui s'écrit dans un fichier .sql du projet est un modèle : du code qui reste, versionné par Git, que dbt rebâtit dans l'entrepôt à chaque dbt run, et que vous livrez. Ce qui se tape dans le notebook est une question posée à l'entrepôt, pour lire, compter, vérifier : le notebook ouvre l'entrepôt en lecture seule, ne change rien, et ses requêtes ne font pas partie du projet. En entreprise, la séparation est la même : le projet dbt dans Git d'un côté, une console SQL ou un notebook de l'autre.

Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dir /b
REM Attendu : 01_de, plateforme et sources.
```

set DBT_PROJECT_DIR=02_ae dit à dbt de chercher son projet dans 02_ae : la fenêtre reste dans Projet_Brulerie, à côté de l'entrepôt. La variable ne vit que dans cette fenêtre. Une fenêtre neuve l'oublie, et chaque cours la redonne.

Le nom n'est pas inventé : DBT_PROJECT_DIR est la variable que dbt lit à la place de son option --project-dir, et sa documentation la donne sous ce nom. D'autres options de dbt ont ainsi leur variable, DBT_ suivi du nom de l'option : DBT_PROFILES_DIR remplace --profiles-dir. Sans la variable, chaque commande devrait finir par --project-dir 02_ae.

Étape 1 — Le dépôt et le projet · STAGING

Un projet dbt est un dossier versionné par Git, qui porte trois sortes de fichiers :

- un seul fichier de projet, dbt_project.yml, qui règle le projet entier ;
- sous models, les modèles, un fichier .sql par vue ou par table, et les fichiers YAML qui les décrivent (étape 2) ;

© 2026 iAlexMG. Tous droits réservés.

- plus tard, au cours 05, les tests.

La connexion à l'entrepôt n'en fait pas partie : elle vit dans un second fichier YAML, `profiles.yml`, hors du dépôt, parce qu'en entreprise elle porte un utilisateur et un mot de passe, qui ne vont jamais dans Git. Deux fichiers YAML règlent donc dbt, l'un dans le dépôt, l'autre à côté :

Fichier	Où il vit	Ce qu'il dit
`dbt_project.yml`	dans `02_ae`, versionné	le nom du projet, le profil de connexion à lire, comment bâtir chaque dossier de modèles
`profiles.yml`	dans `Projet_Brulerie`, hors du dépôt	comment se connecter : le moteur, et le fichier de l'entrepôt

1.1 Créer le dépôt, puis son `.gitignore`.

```
git init 02_ae
REM Attendu : Initialized empty Git repository in …/Projet_Brulerie/02_ae/.git/
notepad 02_ae\.gitignore
```

Le Bloc-notes propose de créer le fichier : accepter. Coller, enregistrer, fermer :

```
# Ce que dbt refait à chaque commande : Git ne le garde pas
target/
logs/
```

dbt écrit dans `target/` le SQL qu'il a réellement envoyé à l'entrepôt, et dans `logs/` son journal. Il les refait à chaque commande.

1.2 Écrire le fichier du projet.

```
notepad 02_ae\dbt_project.yml
```

Le fichier est en YAML : des paires `clé: valeur`, et une indentation faite d'espaces, jamais de tabulations, qui dit ce qui est rangé sous quoi. Le nom d'un projet dbt ne peut pas commencer par un chiffre : le dépôt s'appelle `02_ae`, le projet `brulerie`. `profile` nomme la connexion à lire dans `profiles.yml`. Sous `models`, chaque dossier de modèles reçoit sa matérialisation, la forme que dbt donne au résultat dans l'entrepôt. dbt en connaît quatre :

Matérialisation	Ce que dbt bâtit	Dans ces cours
`view`	une vue : la requête, gardée sous un nom et relancée à chaque lecture	le `STAGING`, ici
`table`	une table : le résultat, calculé à chaque `dbt run` et gardé	les `MARTS`, au cours 04

© 2026 iAlexMG. Tous droits réservés.

Matérialisation	Ce que dbt bâtit	Dans ces cours
`incremental`	une table qui ne reçoit, à chaque passe, que les lignes nouvelles	le fait des ventes, au cours 07
`ephemeral`	rien dans l'entrepôt : dbt colle la requête dans celle des modèles qui la lisent	aucun

`view` est ce que dbt bâtit quand rien n'est dit. Le `STAGING` en vues est le choix courant, celui que recommande dbt Labs, l'éditeur de dbt : une vue ne copie rien et suit la `RAW`. Une équipe le passe en tables le jour où relire la `RAW` à chaque requête coûte trop cher. Certains entrepôts en ajoutent une cinquième, `materialized_view`, une vue qui garde son résultat ; DuckDB n'en a pas. Accepter la création, puis écrire le fichier à partir de ce pseudocode, les trous `____` compris :

```
# 02_ae/dbt_project.yml : le projet dbt de La Brûlerie
name: brulerie
version: "1.0.0"
# 1. le profil de connexion à lire dans profiles.yml
profile: ____

# 2. un dossier de models/ par couche ; le STAGING se matérialise en vues
models:
  brulerie:
    staging:
      +materialized: ____
```

Enregistrer, puis fermer le Bloc-notes.

1.3 Écrire la connexion, dans `Projet_Brulerie`, hors du dépôt.

```
notepad profiles.yml
```

dbt cherche `profiles.yml` dans le dossier d'où on le lance, puis dans un dossier `.dbt` du répertoire personnel : ici, il le trouve dans `Projet_Brulerie`, à côté de `plateforme`. Le profil porte le nom que `dbt_project.yml` lui donne, et une cible, `dev`, la seule du projet ; une entreprise en ajoute une seconde, `prod`, qui écrit ailleurs. Le chemin de l'entrepôt, `plateforme/entrepot/brulerie.duckdb`, se lit depuis le dossier où l'on lance dbt : `Projet_Brulerie`. Accepter la création, puis écrire :

```
# profiles.yml : la connexion de dbt à l'entrepôt partagé, hors du dépôt
____:
  target: dev
  outputs:
    dev:
      # 1. le moteur, et le fichier de l'entrepôt vu depuis Projet_Brulerie
```


© 2026 iAlexMG. Tous droits réservés.

```
    type: ____
    path: ____
```

Enregistrer, puis fermer le Bloc-notes.

1.4 Vérifier le projet et la connexion.

```
dbt debug
REM Attendu : profiles.yml file [OK found and valid], dbt_project.yml file [OK found and valid],
REM Connection test: [OK connection ok], puis All checks passed!
```

Étape 2 — Déclarer la `RAW` · STAGING

Une source est une table que vous n'avez pas fabriquée et que vous vous engagez à ne pas modifier : ici, les quatre tables de la `RAW` que vos vues liront. Une source se déclare le jour où un modèle la lit : les trois journaux attendront les cours 05 et 06. Un modèle lit une source par `{{ source('brulerie', 'raw_tickets') }}`, jamais par son nom. Les doubles accolades sont du Jinja, le langage de gabarits de dbt : avant d'envoyer la requête, dbt les remplace par le vrai nom de la table. Le nom vit alors à un seul endroit, et dbt peut tracer le graphe des modèles depuis leur source. Le Jinja n'a pas de fichier à lui : il s'écrit dans le modèle même, entre les doubles accolades, au milieu du SQL.

Les sources se déclarent dans un fichier YAML, à côté des modèles qui les lisent. DuckDB range ses tables dans le schéma `main` quand on ne lui en nomme pas d'autre ; dbt y écrira aussi vos modèles, et le préfixe du nom dira la couche : `raw_` pour la `RAW`, `stg_` pour le `STAGING`.

2.1 Créer le dossier des modèles du `STAGING`, puis le fichier des sources. `mkdir` crée les dossiers qui manquent sur le chemin.

```
mkdir 02_ae\models\staging
notepad 02_ae\models\staging\_sources.yml
```

Le trait de soulignement en tête du nom range le fichier avant les modèles, dans le dossier. Chaque `name` est le nom exact d'une table de la `RAW`, tel que `show tables` l'a listé au cours 01, à l'étape 1.3 : la description dit ce que garde la table, pas son nom. dbt ne vérifie rien ici, et un nom mal écrit ne se voit qu'au premier modèle qui lit la table. Accepter la création, puis écrire :

```
# 02_ae/models/staging/_sources.yml : la RAW reçue, déclarée une seule fois
sources:
  - name: brulerie
    description: "La RAW du café, tout en texte : chaque export gardé, même deux fois."
    # 1. les tables de la RAW vivent dans le schéma par défaut de DuckDB
    schema: ____
    # 2. une entrée par table qu'un modèle lit
    tables:
      - name: raw_tickets
        description: "Les tickets de la caisse, un par ligne et par export."
      - name: ____
```


© 2026 iAlexMG. Tous droits réservés.

```
    description: "Les lignes des tickets, un produit vendu par ligne et par export."
  - name: ____
    description: "Le catalogue de la caisse au 2025-06-30."
  - name: ____
    description: "Les membres du programme de fidélité au 2025-06-30."
```

Enregistrer, puis fermer le Bloc-notes.

Étape 3 — La première vue, `stg_tickets` · STAGING

Quatre mots, d'abord, que ce cours ne mélange pas :

Le mot	Ce que c'est	Où il sert
le `STAGING`	une couche de l'entrepôt : les modèles qui reprennent la `RAW`, une vue par table	ici, chez l'ingénieur analytique, une fois la `RAW` chargée
un stage, dans Snowflake	un emplacement de fichiers, où l'on dépose un CSV avant de le charger dans une table	avant la `RAW`, chez l'ingénieur de données ; ce cours n'en crée aucun
une vue	une requête enregistrée sous un nom : elle ne garde aucune donnée, et relit la `RAW` chaque fois qu'on la lit	le `STAGING` : toujours à jour avec la `RAW`, sans rien copier
une table	le résultat d'une requête, calculé une fois et gardé jusqu'au prochain `dbt run`	les `MARTS`, au cours 04 : l'étoile, lue souvent, se calcule une fois

Le `STAGING` et le stage se ressemblent par le nom, rien d'autre.

La `RAW` garde les fichiers tels que livrés, et le cours 01 en a vu les pièges : tout est en texte, si bien que `9.78` passe devant `74.93` ; la journée du 12 juin y est deux fois ; des codes comme `O` ou `N` sont à deviner. Le `STAGING` en fait des tables qu'on interroge sans piège, une vue par table de la `RAW`, par quatre gestes :

Le geste	Pour les tickets
Renommer	rien : les noms de la caisse se lisent déjà
Typer	le texte devient des entiers, un horodatage, des montants en `decimal(10, 2)`
Décoder	rien : les codes sont déjà des mots, `PAYE` ou `ANNULE`
Tirer l'état courant	de chaque ticket, garder l'export le plus récent

© 2026 iAlexMG. Tous droits réservés.

Et deux gestes qu'il refuse :

- retirer des tickets selon ce qu'ils disent, par un `where` : les tickets annulés restent dans la vue, puisque décider ce qui compte comme une vente revient à l'`INTERMEDIATE`, au cours 04. Tirer l'état courant ne retire que des copies : chaque ticket reste, une seule fois ;
- joindre une autre table : la jointure est une décision de modélisation, elle vient au cours 04.

Tirer l'état courant, c'est numéroter les exports de chaque ticket, du plus récent au plus ancien, puis ne garder que le premier. Deux outils de SQL le font, l'un après l'autre :

- `row_number() over (partition by … order by … desc)` numérote. Il donne un numéro à chaque ligne, repart à 1 pour chaque valeur de la colonne de `partition by`, ici pour chaque ticket, et suit l'ordre de la colonne de `order by`, ici du plus récent au plus ancien.
- `qualify … = 1` garde. Il ne retient que les lignes numérotées 1, comme `where` filtre sur une colonne.

Le plus récent des exports est celui dont le nom de fichier est le plus grand : la colonne `filename`, vue au cours 01, finit par l'instant de l'export, et `202506130705` vient après `202506121542`.

`row_number()` et `qualify` s'apprennent au Cours 04 de la formation SQL avancé, `Theorie_04_SQL.md`, sections 3 et 7.

Le pseudocode de l'étape 3.1 a trois sortes de trous `____`. Pour chacune, ce qui la remplit, et où le trouver :

Le trou	**Ce qui le remplit**	**Où le trouver**
un type, après `::`	`integer` pour un entier, `timestamp` pour une date et une heure, `decimal(10, 2)` pour un montant au cent	la colonne Type du dictionnaire, traduite par le tableau qui suit ; `ticket_id::integer as ticket_id`, dans le pseudocode, montre la forme
la table lue, après `from`	`{{ source('nom de la source', 'nom de la table') }}`	les deux noms déclarés dans `_sources.yml`, à l'étape 2
les colonnes de `partition by` et de `order by`	la colonne qui dit de quel ticket il s'agit, puis celle qui dit de quel export vient la ligne	le paragraphe au-dessus

Les colonnes à écrire, leur ordre et leur type se lisent à deux endroits, et chacun dit sa part :

Où regarder	**Ce qu'on y trouve**	**Ce qu'on n'y trouve pas**
le dictionnaire, `sources\dictionnaire_brulerie.md`, section `tickets` (le Bloc-notes l'ouvre)	chaque colonne, dans l'ordre du fichier livré, avec son vrai type, colonne Type, et son sens	`filename`, que le chargeur de Data Engineering ajoute

© 2026 iAlexMG. Tous droits réservés.

Où regarder	Ce qu'on y trouve	Ce qu'on n'y trouve pas
le notebook : `describe brulerie.raw_tickets`, comme à l'étape 1.4 du cours 01	les colonnes telles que la `RAW` les garde, dans le même ordre, `filename` en dernier	leur vrai type : la `RAW` garde tout en `VARCHAR`, du texte

Le type à écrire se lit donc au dictionnaire, et se traduit ainsi, pour cette vue comme pour les trois de l'étape 5 :

Type au dictionnaire	Dans le modèle
entier	`::integer`
décimal	`::decimal(10, 2)`
texte AAAA-MM-JJ HH:MM:SS, une date et une heure	`::timestamp`
texte AAAA-MM-JJ, une date seule	`::date`
texte	rien : la colonne est déjà du texte dans la `RAW`, elle monte telle quelle, sauf un code `O` ou `N`, décodé à l'étape 5

Le fichier s'appelle `stg_tickets.sql`, et la vue qu'il donne, `stg_tickets`. Le préfixe `stg_` ne veut pas dire « vue » : il dit la couche, le `STAGING`, comme `raw_` dit la `RAW`. Le cours 04 ajoutera `int_` pour l'`INTERMEDIATE`, puis `dim_` et `fct_` pour les `MARTS`. La forme, vue ou table, c'est `dbt_project.yml` qui la décide, à l'étape 1.2, et elle peut changer sans toucher au nom : passer le dossier `staging` en `table` ferait de `stg_tickets` une table, sous le même nom, et aucun modèle qui la lit n'aurait à changer. Un nom en `vue_` deviendrait faux ce jour-là, et ne dirait pas à quelle couche le modèle appartient. C'est la convention que recommande dbt Labs, l'éditeur de dbt. L'étape 4.4 montre la forme de chaque objet, à part du nom.

3.1 Créer le fichier.

```
notepad 02_ae\models\staging\stg_tickets.sql
```

Accepter la création, puis écrire le modèle à partir de ce pseudocode. Les colonnes gardent leur nom et l'ordre du dictionnaire : les montants passent en `decimal(10, 2)`, les codes, `mode_paiement` et `membre_id` restent du texte, sans conversion (`mode_paiement` dit `DEBIT`, `CREDIT` ou `COMPTANT`, des mots et non des nombres), et la dernière colonne ne prend pas de virgule. La vue aura 13 colonnes. `filename` n'en fait pas partie : elle sert seulement au tri de `qualify`, qui la lit dans la `RAW` même si le `select` ne la garde pas. Dans tous les cours, une ligne `-- puis, à écrire : …` nomme ce que le pseudocode ne donne pas et que vous écrivez vous-même.

```
-- 02_ae/models/staging/stg_tickets.sql : les tickets de la caisse, typés, un par ticket
select
    -- 1. typer : le texte de la RAW devient des nombres et des dates, sous le même nom
    ticket_id::integer as ticket_id,
    horodatage::____ as horodatage,
    type_ticket,
    statut,
```


© 2026 iAlexMG. Tous droits réservés.

```
    ticket_origine::____ as ticket_origine,
    membre_id,
    sous_total::____ as sous_total,
    -- puis, à écrire : tps, tvq, total, mode_paiement, arrondi et encaisse, dans cet ordre
-- 2. lire la RAW par sa source, jamais par son nom
from ____
-- 3. tirer l'état courant : de chaque ticket, garder l'export le plus récent
qualify row_number() over (partition by ____ order by ____ desc) = 1
```

Enregistrer, puis fermer le Bloc-notes.

Étape 4 — Bâtir la vue, puis la lire · STAGING

Une vue n'est pas un fichier de données. Elle existe à trois endroits, sous trois formes : la recette, c'est-à-dire la requête qui dit comment tirer la vue de la `RAW` (quelles colonnes, quel type, quelle copie garder) ; cette recette, compilée par dbt ; et l'objet que l'entrepôt garde sous un nom.

Ce qu'on voit	Où	Ce que c'est
`stg_tickets.sql`	`02_ae\models\staging`, dans le dépôt	la recette : le `select` que vous avez écrit, avec son Jinja, `{{ source('brulerie', 'raw_tickets') }}` à la place du nom de la table
`stg_tickets.sql` compilé	`02_ae\target\compiled\…`, refait à chaque commande	le même `select`, le Jinja remplacé par le nom complet de la table (étape 4.2)
la vue `stg_tickets`	dans l'entrepôt, `plateforme\entrepot\brulerie.duckdb`, à côté de `raw_tickets`	un nom et une requête, rangés dans l'entrepôt : aucune donnée, aucun fichier à elle (étapes 4.4 et 4.5)

`dbt run` lit la recette, la compile, puis demande à l'entrepôt de créer la vue : `create view stg_tickets as (le select compilé)`.

4.1 Lancer dbt, dans la première fenêtre. `dbt run` bâtit chaque modèle du projet, et le projet n'en a qu'un pour l'instant : `stg_tickets.sql`. Un modèle, c'est un fichier `.sql` du dossier `models` ; sa matérialisation dit la forme que dbt lui donne dans l'entrepôt. L'étape 1.2 a mis tout le dossier `staging` en `view` : dbt fait donc de `stg_tickets.sql` une vue, `stg_tickets`. À la fin du cours, le `STAGING` aura quatre modèles, donc quatre vues.

```
dbt run
REM Attendu : 1 of 1 OK created sql view model main.stg_tickets, puis Completed successfully
REM et Done. PASS=1 WARN=0 ERROR=0.
```

4.2 Lire le SQL que dbt a envoyé à l'entrepôt. `type` affiche un fichier dans l'Anaconda Prompt.

© 2026 iAlexMG. Tous droits réservés.

```
type 02_ae\target\compiled\brulerie\models\staging\stg_tickets.sql
REM Attendu : votre modèle, où la source est devenue from "brulerie"."main"."raw_tickets".
```

Dans votre modèle, la table s'écrivait `{{ source('brulerie', 'raw_tickets') }}`. Dans le fichier compilé, dbt l'a remplacée par `"brulerie"."main"."raw_tickets"` : trois noms séparés par des points, du plus grand au plus petit.

Le nom	Ce qu'il désigne
`brulerie`	la base de données, l'entrepôt lui-même
`main`	le schéma, le compartiment de la base où DuckDB range ses tables, celui que `_sources.yml` déclare
`raw_tickets`	la table

L'entrepôt, le fichier `brulerie.duckdb` et la base `brulerie` ne sont pas trois choses, mais une seule, vue de trois côtés :

Le mot	Ce qu'il nomme
l'entrepôt	le rôle : l'endroit où vivent la `RAW` de Data Engineering et vos couches
`plateforme\entrepot\brulerie.duckdb`	le fichier qui le tient, sur le disque : avec DuckDB, un entrepôt tient dans un fichier
`brulerie`	la base de données que contient ce fichier ; DuckDB lui donne le nom du fichier, sans `.duckdb`, et dbt comme le notebook la nomment ainsi

En entreprise, l'entrepôt est un serveur, Snowflake ou BigQuery, et non un fichier, mais la base y porte de même un nom, que dbt écrit en tête de chaque table.

Le nom en trois parties est la seule forme que l'entrepôt comprend : le Jinja ne sort jamais de dbt. Ce fichier compilé montre la requête telle que l'entrepôt l'a reçue : c'est le premier endroit où regarder quand un modèle ne rend pas ce qu'on attend.

4.3 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Laisser cette fenêtre ouverte : la fermer arrête le notebook. Créer un notebook et le renommer « AE Cours 03 », puis attacher l'entrepôt dans la première cellule :

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
-- Attendu : brulerie paraît sous Attached databases, avec raw_tickets et stg_tickets.
```

Le nom `brulerie` n'est pas libre : la vue nomme sa table par le nom de la base, comme l'étape 4.2 l'a montré. Attachée sous un autre nom, elle ne la trouve plus. Si une requête de cette étape s'arrête sur une erreur, le tableau en fin d'étape donne la cause et le geste.

© 2026 iAlexMG. Tous droits réservés.

Chaque bloc SQL du cours se tape dans sa propre cellule, sous celle qui attache l'entrepôt : une cellule qui porte deux requêtes n'affiche que le résultat de la dernière. Une sous-étape qui donne deux blocs demande donc deux cellules.

4.4 Voir ce que contient le fichier de l'entrepôt. `brulerie.duckdb` ne s'ouvre pas dans le Bloc-notes : c'est un fichier binaire, que seul DuckDB sait lire. L'`attach` de la première cellule vient de l'ouvrir, et le panneau de gauche du notebook en montre déjà le contenu, sous Attached databases. Une requête le dit aussi, avec la forme de chaque objet : `information_schema.tables` est le catalogue de la base, la liste de ses tables et de ses vues, que DuckDB tient à jour. En entreprise, la même requête se pose à Snowflake ou à PostgreSQL.

```sql
select table_name, table_type
from information_schema.tables
where table_catalog = 'brulerie'
order by table_type, table_name;
-- Attendu : 10 lignes ; en BASE TABLE, journal et les huit tables de la RAW, de
-- raw_consentements à raw_tickets, puis stg_tickets en VIEW ; deux tables de plus, argent_lots
-- et direct_courbes, si vous avez fait Data Engineering jusqu'au bout.
```

Les tables de la `RAW` gardent des données, la vue n'en garde aucune, et les deux vivent dans le même fichier. `journal` et `raw_courbes` sont à Data Engineering : vous ne les lirez pas. La colonne `table_type` dit la forme de chaque objet ; le préfixe du nom dit sa couche.

4.5 Voir la vue elle-même. `duckdb_views()` est une fonction de DuckDB qui liste les vues d'un entrepôt, avec la requête que chacune garde.

```sql
select view_name, sql
from duckdb_views()
where database_name = 'brulerie';
-- Attendu : une ligne, stg_tickets, et sa requête : CREATE VIEW stg_tickets AS SELECT
-- CAST(ticket_id AS INTEGER) AS ticket_id, …, votre select, réécrit par DuckDB.
```

C'est toute la vue : un nom et cette requête, rangés dans `brulerie.duckdb`. DuckDB l'a réécrite à sa façon, `::integer` en `CAST(… AS INTEGER)` et la source en `brulerie.main.raw_tickets`, mais c'est la vôtre. Chaque fois qu'une requête lit `stg_tickets`, DuckDB relance celle-ci sur `raw_tickets` : c'est pourquoi la vue ne garde aucune donnée.

4.6 Compter les tickets de la vue.

```sql
select count(*) as lignes, count(distinct ticket_id) as tickets
from brulerie.stg_tickets;
-- Attendu : 1527 lignes pour 1527 tickets.
```

4.7 Compter les tickets du 12 juin.

```sql
select count(*) as tickets
from brulerie.stg_tickets
where horodatage::date = '2025-06-12';
-- Attendu : 65, ceux du second export.
```


© 2026 iAlexMG. Tous droits réservés.

4.8 Lire les types de la vue, puis ses cinq premiers tickets, toutes colonnes, en deux cellules.

```
describe brulerie.stg_tickets;
-- Attendu : 13 colonnes ; ticket_id et ticket_origine en INTEGER, horodatage en TIMESTAMP,
-- les six montants en DECIMAL(10,2) ; type_ticket, statut, membre_id et mode_paiement en VARCHAR.
```

```
select *
from brulerie.stg_tickets
limit 5;
-- Attendu : 5 tickets et leurs 13 colonnes, sans erreur ; mode_paiement en mots, comme DEBIT
-- ou CREDIT.
```

Le nombre de colonnes et leurs types se vérifient ici, parce que dbt n'en sait rien : une vue de sept colonnes, ou de quatorze, reste une vue valide, et `dbt run` la bâtit sans se plaindre. Une conversion ratée ne se voit pas non plus au `dbt run` : DuckDB ne calcule que les colonnes qu'une requête lit, et l'erreur n'éclate qu'à la première qui touche la colonne, comme ce `select *`. Moins de 13 colonnes, plus de 13, un type qui ne colle pas ou une erreur : le tableau en fin d'étape donne la cause et le geste.

4.9 Additionner l'encaissé, dans la vue, puis dans la `RAW` en `DOUBLE`, le type que devine un outil quand personne ne décide, en deux cellules. Relancer chacune deux ou trois fois (Ctrl+Entrée).

```
select sum(encaisse) as encaisse
from brulerie.stg_tickets;
-- Attendu : 18666.38, le même à chaque exécution.
```

```
select sum(encaisse::double) as encaisse
from brulerie.raw_tickets;
-- Attendu : presque 19352.36 ; les derniers chiffres peuvent changer d'une exécution à l'autre.
```

La vue a corrigé deux défauts d'un coup. Les 685,98 $ d'écart sont l'encaissé des 58 tickets du premier export du 12 juin, que la `RAW` compte deux fois. Et un `DOUBLE` est un nombre à virgule flottante, une approximation en binaire : additionné dans un ordre qui change, il ne tombe jamais tout à fait juste au cent. Le décimal, lui, compte au cent près.

4.10 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Si tout s'est passé comme prévu, vous n'avez vu aucun des messages du tableau qui suit : il ne sert que lorsqu'une commande ou une requête de l'étape s'arrête, ou rend autre chose que son attendu. Les messages de dbt paraissent dans la première fenêtre, ceux de DuckDB sous la cellule du notebook ; la colonne de gauche dit lequel.

© 2026 iAlexMG. Tous droits réservés.

Ce qui paraît, et où	La cause, et le geste
`No dbt_project.yml found at expected path …\Projet_Brulerie\dbt_project.yml`, dans la première fenêtre	la fenêtre a oublié la variable : relancer `set DBT_PROJECT_DIR=02_ae`. dbt a laissé un dossier `logs` dans `Projet_Brulerie`, sans effet
`IO Error: Cannot open file "…\brulerie.duckdb": Le processus ne peut pas accéder au fichier…`, dans la première fenêtre	le notebook tient l'entrepôt : quitter DuckDB dans la seconde fenêtre, puis relancer dbt
`Catalog "brulerie" does not exist!`, dans le notebook	l'entrepôt est attaché sous un autre nom : le rattacher `as brulerie`
`Binder Error: Referenced column "encaisse" not found in FROM clause!`, dans le notebook	la vue n'a pas toutes ses colonnes (étape 4.8) : quitter DuckDB dans la seconde fenêtre, ajouter à `stg_tickets.sql` les colonnes qui manquent, relancer `dbt run`, puis rouvrir le notebook et relancer sa première cellule
`Conversion Error: Could not convert string "DEBIT" to DECIMAL(10,2)`, ou une autre chaîne, dans le notebook	une colonne de texte a été convertie en montant : `mode_paiement`, qui reste du texte, ou `filename`, qui ne monte pas dans la vue. Quitter DuckDB, retirer la conversion ou la colonne de trop dans `stg_tickets.sql`, relancer `dbt run`, rouvrir le notebook
14 colonnes à l'étape 4.8	une colonne de trop dans `stg_tickets.sql`, le plus souvent `filename` : même geste

Étape 5 — Les trois autres vues · STAGING

Les lignes des tickets, le catalogue et les membres prennent chacun leur vue. `stg_tickets` est venue seule aux étapes 3 et 4, pour faire tout le chemin une fois sur un seul fichier : écrire, `dbt run`, lire le SQL compilé, lire la vue dans le notebook. Une erreur s'y voyait sur un seul modèle. Les trois autres refont le même chemin, ensemble.

Les lignes demandent les mêmes gestes que les tickets ; le catalogue et les membres n'ont qu'un export, au 30 juin, rien à dédoublonner, mais ils montrent les deux gestes qui restent. Comme pour `stg_tickets`, chaque ligne `-- puis, à écrire : …` des pseudocodes nomme des colonnes que vous écrivez vous-même, d'après le dictionnaire et le tableau des types de l'étape 3 : la vue des lignes en aura 10, celle du catalogue 10, celle des membres 5. Une colonne de texte, comme `description`, `code_rabais` ou `code_taxe`, ne se convertit pas : la `RAW` la garde déjà en texte, et la vue la reprend telle quelle.

La vue	Typer	Décoder	Tirer l'état courant	Ne rien faire monter de personnel
`stg_lignes`	les numéros, la quantité, les montants	rien	de chaque ligne de ticket, l'export le plus récent	rien

© 2026 iAlexMG. Tous droits réservés.

La vue	Typer	Décoder	Tirer l'état courant	Ne rien faire monter de personnel
`stg_produits`	le numéro, le prix, les deux dates	`actif` devient `est_actif`	rien : un seul export	rien
`stg_membres`	l'inscription, le solde de points	`infolettre` devient `consent_infolettre`	rien : un seul export	le prénom, le nom, le courriel, le téléphone ; du code postal, le secteur seulement

Décoder, c'est traduire un code en une valeur qui se lit seule. Le catalogue dit `O` ou `N` dans `actif`, et les membres font de même dans `infolettre`. La comparaison `actif = 'O'` rend vrai ou faux, un booléen, et son nom dit ce qu'elle vaut : `est_actif`. Les catégories (`BOISSON_CHAUDE`) se lisent déjà : elles restent telles quelles.

La `RAW` des membres garde leur prénom, leur nom, leur courriel, leur téléphone et leur code postal, le cours 01 l'a montré. Aucune analyse des ventes n'en a besoin, et chaque couche qui les recopie ajoute un endroit d'où ils peuvent fuir. La vue ne les sélectionne pas : ce que le `STAGING` laisse dans la `RAW` n'existe pour aucun modèle au-dessus. Du code postal, elle garde le secteur, ses trois premiers caractères (`substring(code_postal, 1, 3)`), qui dit d'où viennent les membres sans dire où ils habitent.

5.1 Écrire `stg_lignes`, dans la première fenêtre.

```
notepad 02_ae\models\staging\stg_lignes.sql
```

Une ligne se reconnaît à deux colonnes, le numéro du ticket et `ligne`, son rang dans le ticket : l'état courant se tire sur le couple, et `partition by` prend les deux, séparées par une virgule. Accepter la création, puis écrire :

```
-- 02_ae/models/staging/stg_lignes.sql : les lignes des tickets, typées, une par ligne de ticket
select
    -- 1. typer : le texte de la RAW devient des nombres, sous le même nom
    ticket_id::integer as ticket_id,
    ligne::____ as ligne,
    produit_id::integer as produit_id,
    description,
    -- puis, à écrire : quantite, prix_unitaire, rabais, code_rabais, code_taxe et montant, dans cet ordre
-- 2. lire la RAW par sa source
from ____
-- 3. tirer l'état courant : de chaque ligne de ticket, garder l'export le plus récent
qualify row_number() over (partition by ____, ____ order by filename desc) = 1
```

Enregistrer, puis fermer le Bloc-notes.

© 2026 iAlexMG. Tous droits réservés.

5.2 Écrire `stg_produits`.

```
notepad 02_ae\models\staging\stg_produits.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/staging/stg_produits.sql : le catalogue de la caisse, typé et décodé
select
    -- 1. typer, dans l'ordre du dictionnaire
    produit_id::integer as produit_id,
    -- puis, à écrire : nom, categorie, format et code_taxe, tels quels, et prix en montant
    -- 2. décoder : O ou N devient vrai ou faux, sous un nom qui le dit
    actif = '____' as est_actif,
    -- puis, à écrire : saison, telle quelle, et les deux dates, cree_le et modifie_le, en date
-- 3. lire la RAW par sa source ; le catalogue n'a qu'un export, rien à dédoublonner
from ____
```

Enregistrer, puis fermer le Bloc-notes.

5.3 Écrire `stg_membres`.

```
notepad 02_ae\models\staging\stg_membres.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/staging/stg_membres.sql : les membres du programme de fidélité, rien de personnel
select
    membre_id,
    -- 1. du code postal, ne garder que le secteur : ses trois premiers caractères
    substring(code_postal, ____, ____) as secteur_postal,
    -- 2. typer
    inscrit_le::____ as inscrit_le,
    -- puis, à écrire : solde_points, en entier
    -- 3. décoder : O ou N devient vrai ou faux, sous un nom qui le dit
    ____ as consent_infolettre
-- 4. lire la RAW par sa source ; le prénom, le nom, le courriel et le téléphone y restent
from ____
```

Enregistrer, puis fermer le Bloc-notes.

Étape 6 — Bâtir les trois vues, puis les lire · STAGING

6.1 Lancer dbt, dans la première fenêtre.

© 2026 iAlexMG. Tous droits réservés.

```
dbt run
REM Attendu : quatre lignes OK created sql view model, de main.stg_lignes à main.stg_tickets,
REM puis Done. PASS=4 WARN=0 ERROR=0.
```

dbt refait aussi `stg_tickets` : `dbt run` bâtit chaque modèle du projet, chaque fois. S'il s'arrête plutôt sur `depends on a source named 'brulerie.raw_…' which was not found`, le nom de cette table est mal écrit dans `_sources.yml` : le corriger d'après `show tables`, puis relancer.

6.2 Rouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Ouvrir le notebook « AE Cours 03 » et relancer sa première cellule, celle qui attache l'entrepôt, puis celle de l'étape 4.4 : 13 lignes maintenant, les neuf tables de Data Engineering et les quatre vues du `STAGING`.

6.3 Lire les types de la vue des lignes, puis compter ses lignes, en deux cellules.

```
describe brulerie.stg_lignes;
-- Attendu : 10 colonnes ; ticket_id, ligne, produit_id et quantite en INTEGER ; prix_unitaire,
-- rabais et montant en DECIMAL(10,2) ; description, code_rabais et code_taxe en VARCHAR.
```

```
select count(*) as lignes, count(distinct ticket_id) as tickets
from brulerie.stg_lignes;
-- Attendu : 2532 lignes pour 1527 tickets ; la RAW en garde 2626.
```

Les 94 lignes de moins sont celles du premier export du 12 juin, que la `RAW` garde en double.

Les deux cellules vérifient deux choses différentes. Le compte vérifie les lignes, pas les colonnes : une vue à qui il manquerait six colonnes rendrait les mêmes 2 532 lignes. C'est le `describe` qui vérifie que les 10 colonnes sont là, avec leur type : le cours 04 en a besoin, puisqu'il additionne la quantité et les montants de cette vue.

6.4 Lire les types de la vue du catalogue, puis trouver le produit retiré de la vente, en deux cellules. Un booléen se filtre tel quel : `where est_actif`, ou `where not est_actif`.

```
describe brulerie.stg_produits;
-- Attendu : 10 colonnes ; produit_id en INTEGER, prix en DECIMAL(10,2), est_actif en BOOLEAN,
-- cree_le et modifie_le en DATE ; nom, categorie, format, code_taxe et saison en VARCHAR.
```

```
select produit_id, nom, saison
from brulerie.stg_produits
where not est_actif;
-- Attendu : 1 produit, le 111, Latte à l'érable, de la saison ERABLE.
```

Le temps des sucres finit le 30 avril : au 30 juin, la caisse ne le vend plus.

6.5 Lire les colonnes des membres.

© 2026 iAlexMG. Tous droits réservés.

```
describe brulerie.stg_membres;
-- Attendu : 5 colonnes, membre_id, secteur_postal, inscrit_le en TIMESTAMP, solde_points en
-- INTEGER et consent_infolettre en BOOLEAN ; aucune qui permette d'écrire à quelqu'un.
```

Une colonne nommée `inscrit_le_1` : deux colonnes portent le même nom après `as`, et DuckDB renomme la seconde. Chaque colonne garde le sien, `solde_points::integer as solde_points`. Aux étapes 6.3 à 6.5, une colonne qui manque, de trop, mal nommée ou d'un autre type se corrige comme à l'étape 4 : quitter DuckDB, corriger le modèle dans la première fenêtre, relancer `dbt run`, puis rouvrir le notebook.

6.6 Compter les membres qui consentent à l'infolettre.

```
select count(*) as consentants
from brulerie.stg_membres
where consent_infolettre;
-- Attendu : 487.
```

487 des 994 membres de la vue consentent à l'infolettre.

6.7 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 7 — Versionner · STAGING

7.1 Valider les fichiers du dépôt, dans la première fenêtre. `git -C 02_ae` lance Git dans le dépôt sans quitter `Projet_Brulerie`, et `models` y ajoute tout le dossier des modèles.

```
git -C 02_ae add .gitignore dbt_project.yml models
git -C 02_ae commit -m "Cours 03 : le projet dbt et le STAGING"
REM Attendu : 7 files changed.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

`target/` et `logs/` sont ignorés, et `profiles.yml` vit hors du dépôt : Git n'a rien d'autre à garder.

À retenir

- Un projet dbt est un dépôt Git ; la connexion vit à côté, dans `profiles.yml`, jamais dans le dépôt.
- Un modèle lit la `RAW` par `source()`, jamais par son nom : le nom de la table vit à un seul endroit.

© 2026 iAlexMG. Tous droits réservés.

- Le `STAGING` rend une table lisible, une vue par table : il type, décode et tire l'état courant, il laisse le personnel dans la `RAW`, et il ne retire aucun ticket ni ne joint rien.

© 2026 iAlexMG. Tous droits réservés.